FRAMEWORK DOCUMENTATION

Playwright Automation Framework

End-to-end test automation for SauceDemo — architecture, conventions and usage guide.

Project	playwright_project
Test Target	SauceDemo (https://www.saucedemo.com)
Automation Tool	Playwright Test (@playwright/test ^1.64.0)
Language	TypeScript (CommonJS)
Reporting	HTML Report + Allure
Runtime	Node.js v24
Document Version	1.0

1. Overview

This document describes the architecture, structure, configuration, and conventions of the Playwright end-to-end test automation framework used in this project. It is intended for anyone who needs to understand, maintain, extend, or onboard onto the framework.

The framework follows a **modular, layered design** separating test intent from UI interaction details:

- **Tests** express *what* is being verified.
- **Page Objects** express *how* the UI is interacted with.
- **Fixtures** supply *test data and dependencies*.
- **Utils** provide *cross-cutting infrastructure* (environment loading, config).

This separation keeps tests readable and resilient, and localizes change: when the UI changes, only the affected Page Object is edited, not every test.

2. Technology Stack

Component	Package	Version	Purpose
Test runner / automation API	`@playwright/test`	^1.64.0	Test execution, assertions, fixtures, reporters
Browser automation	`playwright`	^1.64.0	Browser engine control
Node type definitions	`@types/node`	^26.6.5	Node.js typings for TypeScript
Allure reporter	`allure-playwright`	^3.4.3	Allure result generation
Allure CLI	`allure-commandline`	^2.36.0	Serving / generating Allure HTML reports

Runtime: Node.js | **Module system:** CommonJS (`"type": "commonjs"` in `package.json`)

3. Directory Structure

```
Playwright_Project/
├── .env                        # Environment variables (URLs, credentials)
├── .gitignore                  # Git ignore rules
├── package.json                # Dependencies and project metadata
├── package-lock.json           # Locked dependency tree
├── playwright.config.ts        # Central Playwright configuration
├── FRAMEWORK.md                # Framework document (Markdown source)
├── FRAMEWORK.html              # Styled HTML template (build input for the PDF)
├── FRAMEWORK.pdf               # Framework document (generated PDF)
│
├── scripts/                    # Documentation tooling
│   └── generate-framework-pdf.js   # Renders FRAMEWORK.html to FRAMEWORK.pdf
│
├── tests/                      # Test specifications (*.spec.ts)
│   ├── navigate.spec.ts        # Login / navigation tests
│   └── inventory.spec.ts       # Product inventory tests
│
├── pages/                      # Page Object Model classes
│   └── LoginPage.ts            # Login page interactions
│
├── fixtures/                   # Test data providers / fixtures
│   └── test.ts                 # User credential provider
│
├── utils/                      # Shared utilities
│   └── envHelpers.ts           # .env parser + ENV export
│
├── playwright-report/          # Generated HTML report (git-ignored)
├── allure-results/             # Raw Allure result data (git-ignored)
└── test-results/               # Traces, screenshots, videos (git-ignored)
```

Layer Responsibilities

Layer	Folder	Responsibility
Test specs	`tests/`	Contain test cases and assertions. Should read like a test script in business language.
Page Objects	`pages/`	Encapsulate locators and UI actions for a single screen. One class per page.
Fixtures / Data	`fixtures/`	Supply test data (users, roles) and reusable setup.
Utilities	`utils/`	Infrastructure helpers unrelated to any single page.
Config	root	Global execution settings (`playwright.config.ts`).

4. Configuration (playwright.config.ts)

The configuration centralizes all execution behaviour.

Key Settings

Setting	Value	Meaning
`testDir`	`./tests`	Root folder Playwright scans for specs.
`fullyParallel`	`true`	Test files and tests run in parallel.
`forbidOnly`	`!!process.env.CI`	Blocks accidental `test.only` on CI.
`retries`	2 on CI, 0 locally	Auto-retry of failed tests on CI only.
`workers`	1 on CI, auto locally	Parallel worker count.
`reporter`	html, allure-playwright	Dual reporting.
`use.baseURL`	`ENV.BASE_URL` (fallback SauceDemo)	Base URL for `page.goto('/')`.
`use.trace`	`on-first-retry`	Captures Playwright trace when a test retries.

Projects

Currently one active project:

- **chromium** — `devices['Desktop Chrome']`

Firefox, WebKit, mobile viewports, and branded browser channels are present but commented out. Uncomment a block to add a browser to the matrix; each `projects` entry runs the entire suite.

Environment Loading

The config imports `ENV` from `utils/envHelpers` and uses `ENV.BASE_URL` as the `baseURL`. A commented-out `dotenv` block shows the alternative approach — the custom `envHelpers` parser is used instead so the framework has **no dependency on `dotenv`**.

5. Environment Variable Handling (utils/envHelpers.ts)

The project uses a **custom `.env` parser** instead of the `dotenv` package.

How It Works

1. `parseEnv()` resolves `.env` against `process.cwd()`.
2. If the file is missing, it returns an empty object (graceful degradation — no crash).
3. Each line is trimmed; blanks and lines starting with `#` are skipped.
4. Lines are split on the **first** `=` so values may contain `=`.
5. Surrounding single/double quotes are stripped from values.
6. Every parsed key is also written to `process.env` **only if not already set** (real environment variables take precedence over `.env`).

7. The parsed map is exported as the constant `ENV`.

Exported API

```
export const ENV: Record<string, string>;
```

Access values as `ENV.KEY_NAME`. Because it is a `Record<string, string>`, unknown keys return `undefined` at runtime — prefer added named helpers if strong typing is required.

Current .env Keys

```
BASE_URL                https://www.saucedemo.com
STANDARD_USER_NAME      standard_user
STANDARD_USER_PASSWORD  secret_sauce
LOCKED_USER_NAME        locked_out_user
LOCKED_USER_PASSWORD    secret_sauce
PROBLEM_USER_NAME       problem_user
PROBLEM_USER_PASSWORD   secret_sauce
```

> **Security note:** `.env` holds credentials and is listed in `.gitignore`. Never commit real secrets. A `.env.example` (recommended addition) should document required keys with placeholder values.

6. Page Object Model (pages/)

Each page is a class encapsulating locators and actions. This keeps selectors in one place and tests free of raw selectors.

Example Class: LoginPage

```
export class LoginPage {
    readonly page: Page;
    readonly usernameInput: Locator;
    readonly passwordInput: Locator;
    readonly loginButton: Locator;

    constructor(page: Page) { ... }

    async navigateToLoginPage(url: string = '/'): Promise<void> { ... }
    async getPageTitle(): Promise<string> { ... }
    async login(username: string, password: string): Promise<void> { ... }
}
```

Conventions

- **Locators are `readonly` class fields**, initialized in the constructor from the injected `Page`. They are defined once and reused.

- **Selector strategy:** prefer user-facing locators — `[data-test="..."]` attributes, `getByRole`, `getByPlaceholder`, `getByText`. Avoid brittle CSS/XPath chains and index-based selection.
- **Action methods** are `async` and return `Promise<void>` (or a typed value). They perform UI interactions only.
- **Validation** — guard clauses throw descriptive errors early (e.g. `login()` throws if credentials are empty) rather than failing opaquely deep in the browser call.
- **One class per page**, named `<PageName>Page`, in its own file under `pages/`.

Adding a New Page Object

1. Create `pages/<Name>Page.ts`.
2. Define `readonly` locators in the constructor.
3. Add `async` action methods with descriptive verb names.
4. Import into the relevant spec and instantiate with the fixture `page`.

7. Fixtures & Test Data (fixtures/test.ts)

Test data is centralized so credentials never appear inline in specs.

User Provider

```
export type UserType = 'standard' | 'lockedOut' | 'problem';

export function getUser(type: UserType): UserCredentials;
```

- `UserType` is a **union of allowed roles** — TypeScript rejects invalid role strings at compile time.
- The `users` map binds each role to `ENV` credentials.
- `getUser()` returns a `{ username, password }` object and **throws early** if credentials are missing, surfacing configuration errors at the source.

Usage in a Test

```
import { getUser } from '../fixtures/test';

const standardUser = getUser('standard');
await loginPage.login(standardUser.username, standardUser.password);
```

This indirection means tests reference **roles**, not literal credential strings, and `.env` key names never leak into test files.

Extending

- Add a new role: extend the `UserType` union **and** the `users` map.

- The `UserCredentials` interface (`{ username: string; password: string }`) is exported for reuse.

8. Test Specifications (tests/)

Specs are named `*.spec.ts` and organized by feature/area. Each uses `test.describe` for grouping and `test.beforeEach` for shared setup.

navigate.spec.ts — Login / Navigation

- Instantiates `LoginPage` in `beforeEach` , navigating via `baseURL` .
- **Standard user:** asserts redirect to `inventory.html` .
- **Locked-out user:** asserts the error banner contains `"locked out"` .
- Demonstrates the full POM + fixture + config chain working together.

inventory.spec.ts — Inventory Behaviour

- Logs in via direct page interactions in `beforeEach` .
- Tests: product count (6), sort by price ascending, add to cart, remove from cart.
- Demonstrates data-driven assertions (mapping displayed prices to numbers and comparing against a sorted copy).

Test Authoring Conventions

- **Import from** `@playwright/test` for `test` and `expect` .
- Use `beforeEach` for setup shared across a `describe` block.
- Prefer **user-visible assertions** (`toBeVisible` , `toHaveText` , `toHaveURL` , `toHaveCount`).
- Use `expect` with `await` ; every assertion is auto-retrying.
- Keep each test focused on one behaviour; use `describe` for feature grouping.
- Avoid hard-coded URLs in tests — use `baseURL` from config.

9. Running the Framework

> **Note:** `package.json` currently defines **no** `scripts` . Run Playwright directly via `npx` , or add scripts (recommended — see below).

Common Commands

Goal	Command
Run all tests	`npx playwright test`
Run a single file	`npx playwright test tests/navigate.spec.ts`
Run by title	`npx playwright test -g "Login with standard user"`
Run one project	`npx playwright test --project=chromium`
Headed mode	`npx playwright test --headed`
Debug mode	`npx playwright test --debug`
UI mode	`npx playwright test --ui`
Show HTML report	`npx playwright show-report`
Install browsers	`npx playwright install`

Allure Reporting

Goal	Command
Generate Allure report	`npx allure generate allure-results --clean -o allure-report`
Open Allure report	`npx allure open allure-report`

Raw results are written to `allure-results/`; the generated site goes to `allure-report/`.

Recommended scripts (add to package.json)

```
"scripts": {
  "test": "playwright test",
  "test:headed": "playwright test --headed",
  "test:ui": "playwright test --ui",
  "test:chromium": "playwright test --project=chromium",
  "report": "playwright show-report",
  "allure:generate": "allure generate allure-results --clean -o allure-report",
  "allure:open": "allure open allure-report"
}
```

Regenerating this document

This document is generated from `FRAMEWORK.html` using the project's own Playwright Chromium:

```
node scripts/generate-framework-pdf.js
```

The script resolves the placeholder tokens used for angle brackets inside code samples and writes `FRAMEWORK.pdf`.

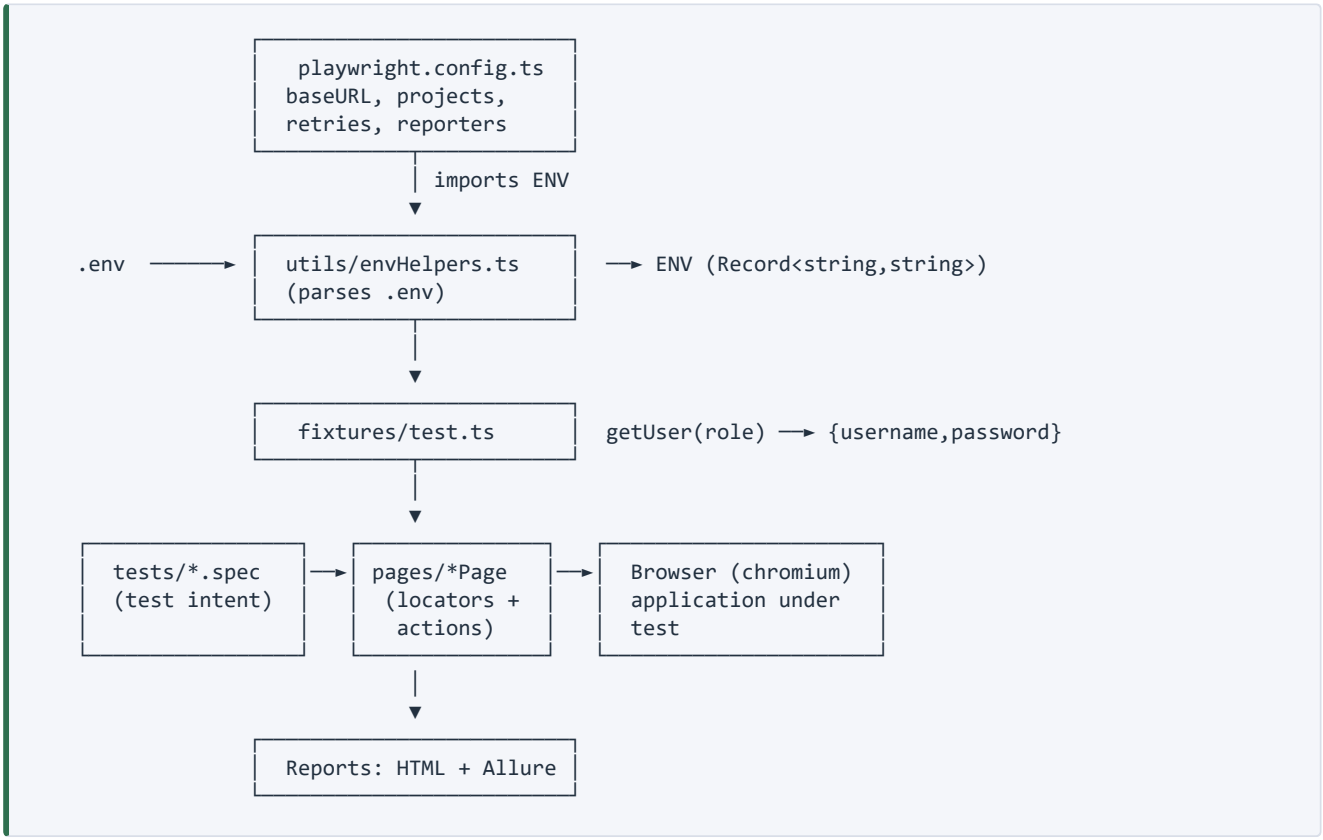
10. Reporting & Artifacts

Output	Location	Contents
HTML report	`playwright-report/`	Interactive results, per-test steps, traces
Allure raw	`allure-results/`	JSON results + attachments for Allure
Test artifacts	`test-results/`	Traces, screenshots, videos for failures

Trace capture is set to `on-first-retry`. When a test fails and retries, Playwright records a trace openable with `npx playwright show-trace <trace.zip>` — invaluable for post-mortem debugging on CI.

All three output directories are git-ignored and regenerated per run.

11. Framework Data Flow



Flow summary: Config loads `ENV` → fixtures use `ENV` to build role-based credentials → specs consume fixtures and drive Page Objects → Page Objects act on the browser → results flow to reporters.

12. Coding Standards & Conventions

File & Naming

- Test specs: `*.spec.ts`, lowerCamelCase topic (`navigate.spec.ts`).
- Page objects: `PascalCase` class, file matches class (`LoginPage.ts`).
- Utilities: lowerCamelCase file (`envHelpers.ts`).
- One primary export per file.

TypeScript

- Prefer explicit types; avoid `any`.
- Use union types for constrained value sets (see `UserType`).
- Guard clauses throw with actionable messages.
- Keep page objects declarative and free of business assertions.

Locators

- Prefer `data-test` attributes and Playwright's user-facing locators over CSS/XPath.
- Never use index-based locators where a stable attribute exists.
- Define locators once (constructor), never inline in multiple methods.

Tests

- One behaviour per test; descriptive `test('...')` titles.
- Shared setup in `beforeEach`.
- Assertions via auto-retrying `expect`.

13. Extending the Framework

Add a New Page

1. Create `pages/<Name>Page.ts` with `readonly` locators and `async` actions.
2. Reference it from specs via the `page` fixture.

Add a New Test Data Set

1. Add keys to `.env`.
2. Extend the relevant fixture map / type.

Add a New Browser

Uncomment the desired block in `projects` **or** add:

```
{ name: 'firefox', use: { ...devices['Desktop Firefox'] } },
```

Add a New Test Layer (e.g. API checks)

- Create `tests/api/` and use Playwright's `request` fixture.
- Keep a separate `api/` client folder for endpoint wrappers.

Recommended Future Enhancements

Enhancement	Benefit
`package.json` scripts	One-word commands (`npm test`).
`.env.example`	Documents required variables without secrets.
Playwright fixtures (`test.extend`)	Auto-inject Page Objects instead of manual `new` .
CI pipeline (GitHub Actions)	Automated runs on push/PR with Allure publishing.
`storageState` auth reuse	Skip UI login per test; faster suites.
ESLint + Prettier	Consistent style enforcement.
`expect` timeout / retry tuning	Stability on slow environments.

14. Troubleshooting

Symptom	Likely Cause	Resolution
`Missing credentials for user type "..."`	`.env` key absent or blank	Add the key(s) to `.env` .
Tests hit wrong site	`BASE_URL` wrong or unset	Check `.env` ; fallback is SauceDemo.
Browser launch failure	Browsers not installed	`npx playwright install` .
Flaky tests on CI	Timing / parallelism	Rely on `retries: 2` ; tighten waits using auto-retrying assertions.
Allure report empty	`allure-results/` missing	Run tests first; then `allure generate` .
Locators not found	UI changed	Update the affected Page Object only.

15. Summary

This framework applies the **Page Object Model**, a **centralized configuration**, an **environment abstraction layer**, and a **role-based test-data provider** to produce readable, maintainable, and debuggable Playwright tests in TypeScript. Tests express intent; Page Objects own UI detail; `envHelpers` and `fixtures` own data; and dual HTML + Allure reporting provides rich diagnostics. Extending the suite follows clear conventions: add a `pages/*` class for new screens, a fixture for new data, and a `*.spec.ts` for new behaviour — the rest of the framework stays untouched.
